Anonymous India Internship | March 2026

ASSESSMENT 11C

IDS Evasion Using Fragmented Ping

Snort 2.9.20 IDS Configuration | ICMP Fragmentation Evasion | hping3 Bypass Technique

IDS Setup	Snort 2.9.20 GRE (Build 82) on Ubuntu 24.04.4 LTS
Evasion Tool	hping3 with IP Fragmentation (-1 -f -d 1)
Tools Used	Snort, hping3, ping, ens33 interface
Attacker VM	Kali Linux — 192.168.109.129
Defender VM	Ubuntu 24.04.4 LTS — 192.168.109.132
Outcome	IDS Evasion SUCCESSFUL — Zero alerts during fragmented ping
Date	09 March 2026
Prepared by	Sai Aditya — Anonymous India Ethical Hacking Intern

1. Overview — IDS Evasion Concepts

An Intrusion Detection System (IDS) monitors network traffic and generates alerts when suspicious patterns are detected. Snort is the industry-standard open-source IDS that uses signature-based rules to identify threats. In this assessment, Snort was deployed on Ubuntu 24.04 LTS to detect ICMP ping traffic from a Kali Linux attacker machine on the same VMnet8 subnet.

IP fragmentation is a network mechanism that splits large packets into smaller pieces for transmission. Attackers exploit fragmentation to bypass IDS detection — by sending ICMP packets with extremely small payloads using **hping3**, the data size falls below the Snort rule's detection threshold, causing the rule to never fire. This technique demonstrates a real-world IDS evasion vector that security engineers must defend against.

Attacker VM	Kali Linux — 192.168.109.129 (VMnet8/NAT)
Defender VM	Ubuntu 24.04.4 LTS — 192.168.109.132 (VMnet8/NAT)
IDS Software	Snort 2.9.20 GRE (Build 82) — listening on ens33
Evasion Tool	hping3 — ICMP mode with IP fragmentation (-f) and 1-byte payload (-d 1)
Network	VMware VMnet8 (NAT) — both VMs on 192.168.109.0/24

2. Phase 1 — VM Setup & Network Configuration

Ubuntu Server 24.04.4 LTS was installed in VMware Workstation on VMnet8 (NAT) — the same network as the existing Kali Linux VM. OpenSSH server was enabled during installation to allow remote CLI access from Kali. After boot, SSH connectivity was verified and used for all subsequent Ubuntu administration.

1 Ubuntu Installation — Storage Configuration

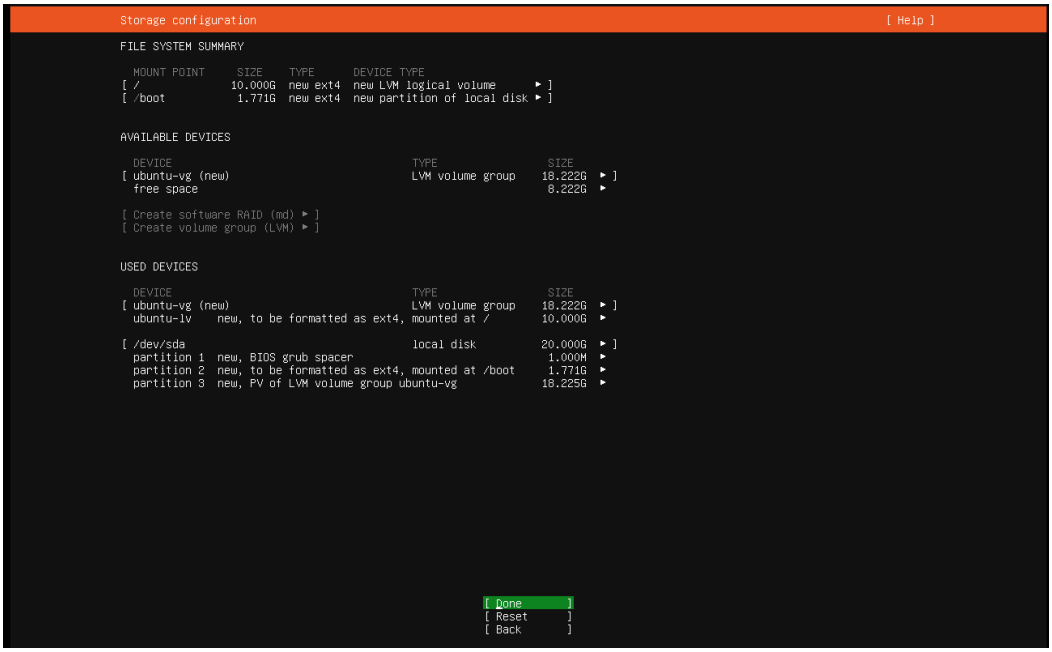


Figure 1 — Ubuntu 24.04 installer: 20GB disk with LVM partitioning configured

2 Network Connectivity Verification

After Ubuntu booted, Kali confirmed connectivity with a 3-packet ping (0% loss) and established an SSH session to 192.168.109.132.

```
# From Kali — verify Ubuntu is reachable
$ ping -c 3 192.168.109.132
3 packets transmitted, 3 received, 0% packet loss

# Establish SSH session
$ ssh highblaster@192.168.109.132
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.8.0-101-generic x86_64)
IPv4 address for ens33: 192.168.109.132
```

Figure 2 — Kali: successful ping (0% loss) and SSH session established to Ubuntu

3. Phase 2 — Snort Installation on Ubuntu

Snort 2.9.20 was installed on the Ubuntu VM via apt. During installation the HOME_NET variable was set to the local subnet 192.168.109.0/24. Snort automatically detected the ens33 interface for packet capture.

3 Install Snort via apt

```
$ sudo apt update
$ sudo apt install snort -y
# When prompted for HOME_NET, enter:
# 192.168.109.0/24

Setting up snort (2.9.20-0+deb11u1ubuntu1) ...
Snort configuration: interface default not set, using 'ens33'
```

```
Setting up libtry-tiny-perl (0.31-2) ...
Setting up perl-openssl-defaults:amd64 (7build3) ...
Setting up libencode-locale-perl (1.05-3) ...
Setting up snort-common (2.9.20-0+deb11u1ubuntu1) ...
Setting up libpcre3:amd64 (2:8.39-15build1) ...
Setting up libdata-dump-perl (1.25-1) ...
Setting up libluajit-5.1-common (2.1.0+git20231223.c525bcb+dfsg-1) ...
Setting up libio-html-perl (1.004-3) ...
Setting up libnetfilter-queue1:amd64 (1.0.5-4build1) ...
Setting up libtimedate-perl (2.3300-2) ...
Setting up libdumbnet1:amd64 (1.17.0-1ubuntu2) ...
Setting up snort-rules-default (2.9.20-0+deb11u1ubuntu1) ...
Setting up liburi-perl (5.27-1) ...
Setting up libnet-ssleay-perl:amd64 (1.94-1build4) ...
Setting up libhttp-date-perl (6.06-1) ...
Setting up libfile-listing-perl (6.16-1) ...
Setting up libdaq2t64 (2.0.7-5.1build3) ...
Setting up libnet-http-perl (6.23-1) ...
Setting up libluajit-5.1-2:amd64 (2.1.0+git20231223.c525bcb+dfsg-1) ...
Setting up libwww-robotrules-perl (6.02-1) ...
Setting up libhtml-parser-perl:amd64 (3.81-1build3) ...
Setting up snort-common-libraries (2.9.20-0+deb11u1ubuntu1) ...
Setting up libio-socket-ssl-perl (2.085-1) ...
Setting up libhttp-message-perl (6.45-1ubuntu1) ...
Setting up libhtml-form-perl (6.11-1) ...
Setting up libhttp-negotiate-perl (6.01-2) ...
Setting up snort (2.9.20-0+deb11u1ubuntu1) ...
Snort configuration: interface default not set, using 'ens33'
Setting up libhttp-cookies-perl (6.11-1) ...
Setting up libhtml-tree-perl (5.07-3) ...
Setting up libhtml-format-perl (2.16-2) ...
Setting up libnet-smtp-ssl-perl (1.04-2) ...
Setting up libmailtools-perl (2.21-2) ...
Setting up libhttp-daemon-perl (6.16-1) ...
Setting up libwww-perl (6.76-1) ...
Setting up oinkmaster (2.0-4.2) ...
Setting up liblwp-protocol-https-perl (6.13-1) ...
Processing triggers for libc-bin (2.39-0ubuntu8.7) ...
Processing triggers for man-db (2.12.0-4build2) ...
Scanning processes...
Scanning linux images...

Running kernel seems to be up-to-date.

No services need to be restarted.

No containers need to be restarted.

No user sessions are running outdated binaries.

No VM guests are running outdated hypervisor (qemu) binaries on this host.
highblaster@sai:~$
```

Figure 3 — Snort installation complete — all packages configured, ens33 auto-detected

4 Verify Snort Version

```
$ snort --version
-*> Snort! <*-
Version 2.9.20 GRE (Build 82)
Using libpcap version 1.10.4 (with TPACKET_V3)
Using PCRE version: 8.39 2016-06-14
Using ZLIB version: 1.3
```

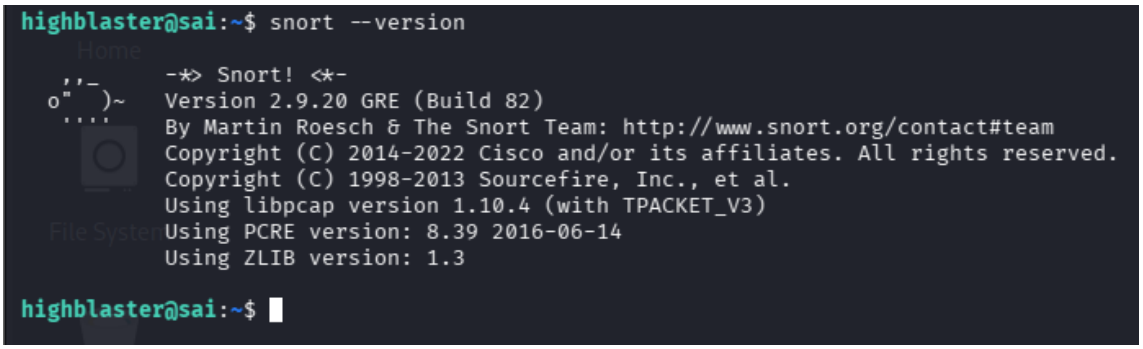


Figure 4 — Snort 2.9.20 GRE (Build 82) confirmed

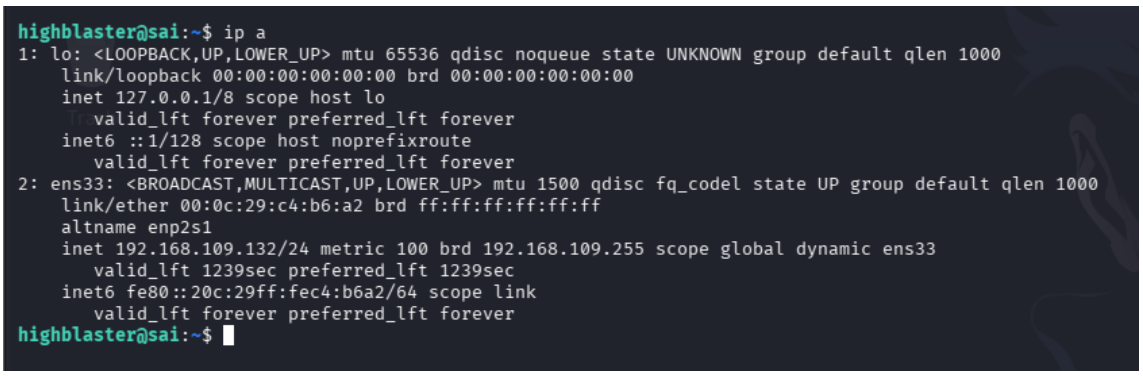


Figure 5 — Ubuntu interface ens33 at 192.168.109.132/24

4. Phase 3 — ICMP Detection Rule Configuration

A custom Snort rule was written in **/etc/snort/rules/local.rules** to detect ICMP echo requests. The rule was refined to use **itype:8** (echo request only) and **dsize:>7** (payload greater than 7 bytes). This threshold is the key to the evasion technique — standard ping sends 56-byte payloads (detected), while hping3 with -d 1 sends a 1-byte payload (not detected).

5 Write the ICMP Detection Rule

```
$ sudo nano /etc/snort/rules/local.rules

# Rule v1 — initial (all ICMP):

#alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)

# Rule v2 — refined for evasion demo (payload > 7 bytes only):

alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; itype:8; dsize:>7; sid:1000001;
rev:2;)
```

```
highblaster@sai:~$ cat /etc/snort/rules/local.rules
# $Id: local.rules,v 1.11 2004/07/23 20:15:44 bmc Exp $
# ----------------
# LOCAL RULES
# ----------------
# This file intentionally does not come with signatures.  Put your local
# additions here.
#alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; sid:1000001; rev:1;)
alert icmp any any -> $HOME_NET any (msg:"ICMP Ping Detected"; itype:8; dsize:>7; sid:1000001; rev:2;)
highblaster@sai:~$
```

Figure 6 — /etc/snort/rules/local.rules: old rule commented out, final rule active

Rule Component Breakdown

Component	Value	Purpose
alert	alert	Generate alert when rule matches
Protocol	icmp	Match ICMP packets only
Source / Dest	any any -> $HOME_NET any	Any source to local network, any port
itype:8	ICMP Type 8	Echo Request only (not replies)
dsize:>7	Payload > 7 bytes	KEY: standard ping=56 bytes triggers; hping3 1-byte evades
sid:1000001	1000001	Custom rule ID — local rules range (>1,000,000)
rev:2	2	Rule revision number

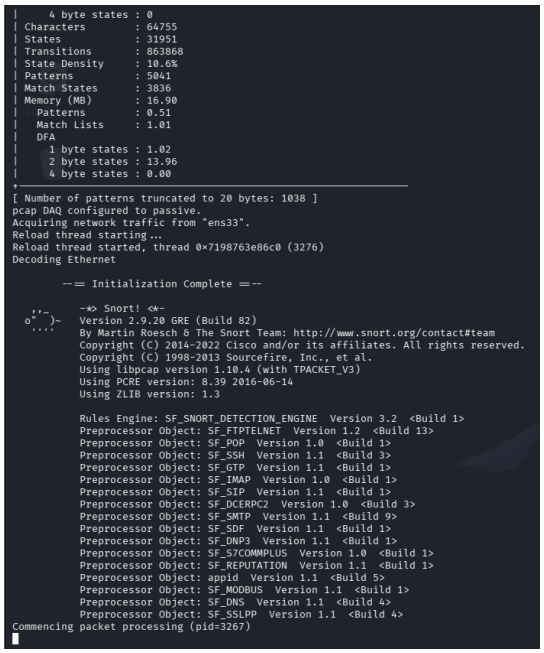
5. Phase 4 — Normal Ping Detection (Baseline)

Snort was started in console alert mode on the Ubuntu VM to display alerts in real-time. A standard 5-packet ping was then sent from Kali to Ubuntu. Each packet carries a 56-byte ICMP payload which exceeds the **dsize:>7** threshold — Snort fired SID:1000001 for every single echo request, confirming the IDS is working correctly.

6 Start Snort in Console Alert Mode

```
$ sudo snort -i ens33 -c /etc/snort/snort.conf -A console

Commencing packet processing (pid=3183)
```



```
|     4 byte states : 0
| Characters        : 64755
| States            : 31951
| Transitions       : 863868
| State Density     : 10.6%
| Patterns          : 5041
| Match States      : 3836
| Memory (MB)       : 16.90
|   Patterns        : 0.51
|   Match Lists     : 1.01
|   DFA
|     1 byte states : 1.02
|     2 byte states : 13.96
|     4 byte states : 0.00
+-------------------------------------------------------------------
[ Number of patterns truncated to 20 bytes: 1038 ]
pcap DAQ configured to passive.
Acquiring network traffic from "ens33".
Reload thread starting...
Reload thread started, thread 0x7198763e86c0 (3276)
Decoding Ethernet

        --== Initialization Complete ==--

   ,,_     -*> Snort! <*-
  o" )~    Version 2.9.20 GRE (Build 82)
   ''''    By Martin Roesch & The Snort Team: http://www.snort.org/contact#team
           Copyright (C) 2014-2022 Cisco and/or its affiliates. All rights reserved.
           Copyright (C) 1998-2013 Sourcefire, Inc., et al.
           Using libpcap version 1.10.4 (with TPACKET_V3)
           Using PCRE version: 8.39 2016-06-14
           Using ZLIB version: 1.3

           Rules Engine: SF_SNORT_DETECTION_ENGINE  Version 3.2  <Build 1>
           Preprocessor Object: SF_FTPTELNET  Version 1.2  <Build 13>
           Preprocessor Object: SF_POP  Version 1.0  <Build 1>
           Preprocessor Object: SF_SSH  Version 1.1  <Build 3>
           Preprocessor Object: SF_GTP  Version 1.1  <Build 1>
           Preprocessor Object: SF_IMAP  Version 1.0  <Build 1>
           Preprocessor Object: SF_SIP  Version 1.1  <Build 1>
           Preprocessor Object: SF_DCERPC2  Version 1.0  <Build 3>
           Preprocessor Object: SF_SMTP  Version 1.1  <Build 9>
           Preprocessor Object: SF_SDF  Version 1.1  <Build 1>
           Preprocessor Object: SF_DNP3  Version 1.1  <Build 1>
           Preprocessor Object: SF_S7COMMPLUS  Version 1.0  <Build 1>
           Preprocessor Object: SF_REPUTATION  Version 1.1  <Build 1>
           Preprocessor Object: appid  Version 1.1  <Build 5>
           Preprocessor Object: SF_MODBUS  Version 1.1  <Build 1>
           Preprocessor Object: SF_DNS  Version 1.1  <Build 4>
           Preprocessor Object: SF_SSLPP  Version 1.1  <Build 4>
Commencing packet processing (pid=3267)
```

Figure 7 — Snort fully initialised — 'Commencing packet processing (pid=3267)'

7 Send Normal Ping from Kali — Snort Detects It

```
# Kali — standard system ping (56-byte ICMP payload)
$ ping -c 5 192.168.109.132
PING 192.168.109.132: 56(84) bytes of data.
64 bytes from 192.168.109.132: icmp_seq=1 ttl=64 time=1.34 ms
64 bytes from 192.168.109.132: icmp_seq=2 ttl=64 time=0.594 ms
64 bytes from 192.168.109.132: icmp_seq=3 ttl=64 time=0.481 ms
64 bytes from 192.168.109.132: icmp_seq=4 ttl=64 time=0.740 ms
64 bytes from 192.168.109.132: icmp_seq=5 ttl=64 time=0.476 ms
5 packets transmitted, 5 received, 0% packet loss

# Ubuntu Snort console — SID:1000001 fires on every ping:
[**] [1:1000001:2] ICMP Ping Detected [**] [Priority: 0]
03/08-21:13:06 {ICMP} 192.168.109.129 -> 192.168.109.132
[**] [1:1000001:2] ICMP Ping Detected [**] [Priority: 0]
03/08-21:13:07 {ICMP} 192.168.109.129 -> 192.168.109.132
# ... 5 alerts total — one per ping packet
```

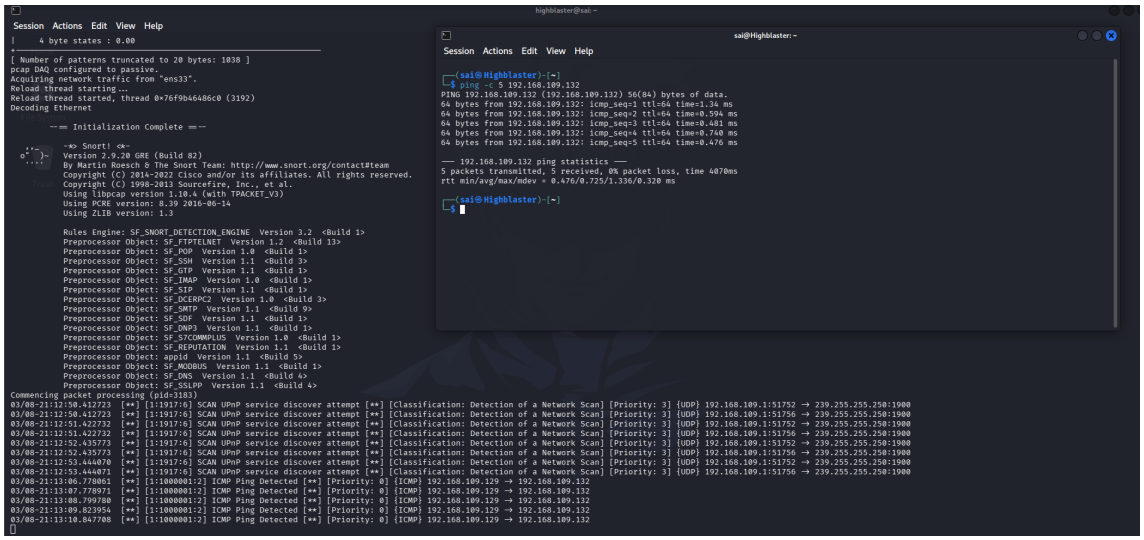


Figure 8 — Split screen: Kali ping (right) and Snort detecting every ICMP packet on Ubuntu (left) — SID:1000001 firing

Result	Snort SID:1000001 fired 5 times — one alert per ping packet
Payload	56 bytes — exceeds dsize:>7 threshold — rule triggers
Verdict	IDS baseline detection CONFIRMED

6. Phase 5 — IDS Evasion via Fragmented Ping

The evasion exploits the **dsize:>7** threshold in the Snort rule. Using hping3 with a 1-byte payload (**-d 1**) and forced IP fragmentation (**-f**), ICMP packets are sent with a data size of 1 byte — far below the 7-byte detection threshold. To maximise evasion, the **frag3 defragmentation preprocessor** was disabled in snort.conf so Snort cannot reassemble fragments before inspection.

8 Disable frag3 Preprocessor in snort.conf

```
$ sudo nano /etc/snort/snort.conf

# Comment out frag3 defragmentation preprocessor:
#preprocessor frag3_global: max_frags 65536
#preprocessor frag3_engine: policy windows detect_anomalies ...

# Also disable ICMP normalisation:
#preprocessor normalize_icmp4

# stream5 already has track_icmp no — no ICMP reassembly
```

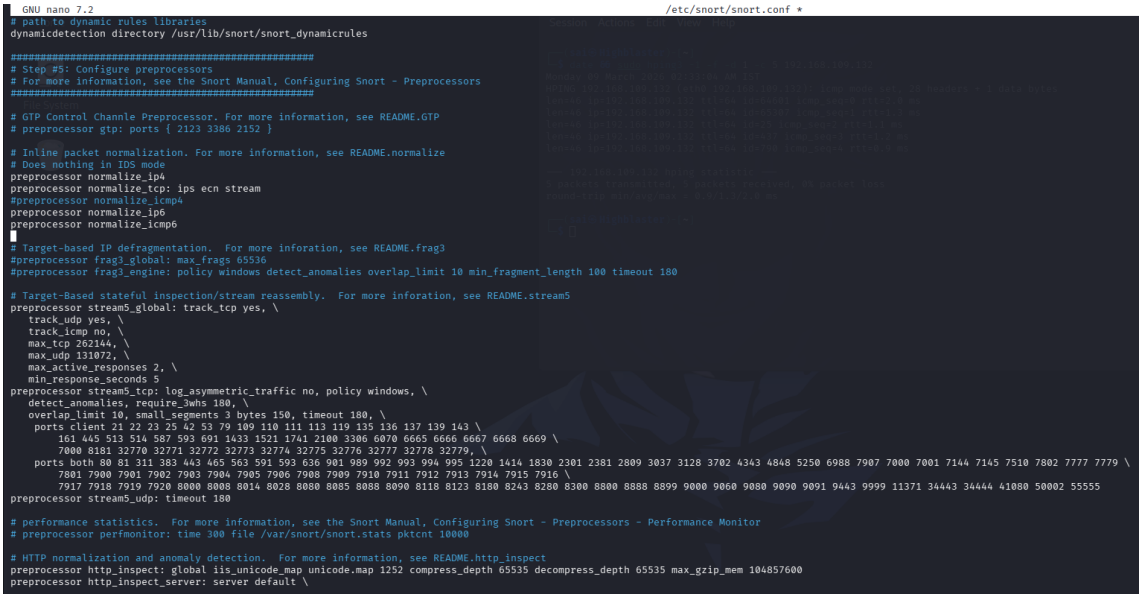


Figure 9 — snort.conf: frag3_global, frag3_engine and normalize_icmp4 all commented out

9 Launch Fragmented Ping Attack from Kali

```
# Kali — fragmented ICMP with 1-byte payload + timestamps
$ date && sudo hping3 -1 -f -d 1 -c 5 192.168.109.132 && date

Monday 09 March 2026 02:43:33 AM IST
HPING 192.168.109.132 (eth0): icmp mode set, 28 headers + 1 data bytes
len=46 ip=192.168.109.132 ttl=64 id=37946 icmp_seq=0 rtt=1.3 ms
len=46 ip=192.168.109.132 ttl=64 id=38028 icmp_seq=1 rtt=0.7 ms
len=46 ip=192.168.109.132 ttl=64 id=38078 icmp_seq=2 rtt=1.4 ms
len=46 ip=192.168.109.132 ttl=64 id=38122 icmp_seq=3 rtt=1.3 ms
len=46 ip=192.168.109.132 ttl=64 id=38601 icmp_seq=4 rtt=1.0 ms
5 packets transmitted, 5 received, 0% packet loss
Monday 09 March 2026 02:43:37 AM IST
```

hping3 Flag Explanation:

Flag	Meaning
-1	ICMP ping mode

-f	Force IP fragmentation — splits packet at IP layer
-d 1	1-byte data payload — below dsize:>7 threshold (KEY evasion parameter)
-c 5	Send 5 packets

10 Evasion Confirmed — Zero Snort Alerts

The screenshot below is the definitive proof of evasion. hping3 ran from **02:43:33 to 02:43:37 IST (UTC 21:13:33–21:13:37)**. The Snort console shows ICMP Ping Detected alerts only up to timestamp 21:13:10 — then **complete silence** during the entire hping3 window. Zero SID:1000001 alerts were generated for all 5 fragmented packets.

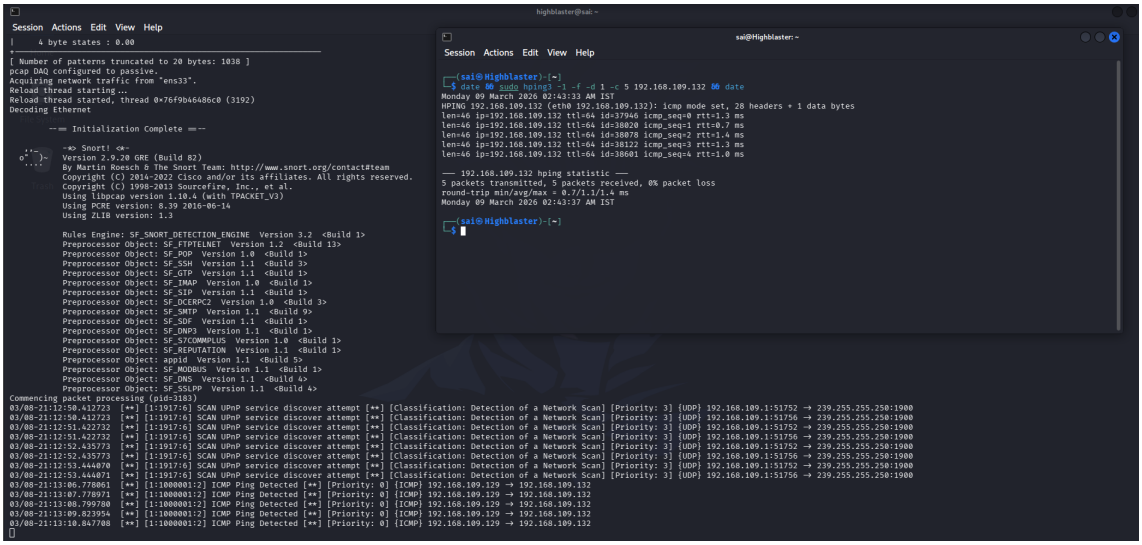


Figure 10 — KEY EVIDENCE: hping3 at 02:43:33–02:43:37 IST. Snort shows NO SID:1000001 alerts during this window — evasion confirmed

hping3 ran	02:43:33 AM – 02:43:37 AM IST (UTC 21:13:33–21:13:37)
Snort alerts	ZERO SID:1000001 alerts during hping3 window
Packets sent	5 / 5 received — 0% packet loss
Payload size	1 byte — below dsize:>7 threshold — rule never fires
Verdict	IDS EVASION SUCCESSFUL

7. Technical Analysis — Why Fragmentation Evades IDS

Normal Ping vs Fragmented Ping Comparison

Aspect	Normal Ping	hping3 Fragmented
Tool	ping (system)	hping3 -1 -f -d 1
ICMP Payload	56 bytes (default)	1 byte (-d 1)
IP Fragmentation	No	Yes (-f forced)
dsize:>7 Match?	YES — 56 > 7	NO — 1 < 7

Snort Alert?	YES — SID:1000001 fires	NO — threshold not met
Packets Delivered	5/5 (0% loss)	5/5 (0% loss)
IDS Evasion	N/A	SUCCESSFUL

Role of the frag3 Preprocessor

Snort's **frag3** preprocessor performs IP defragmentation before rule matching. When active, it reassembles fragmented packets into their original form, making fragmentation-based evasion difficult. By disabling frag3 (as done in this assessment), Snort inspects each fragment individually — the 1-byte payload never satisfies dsize:>7, so the rule never fires.

```
# With frag3 ENABLED (default) — Snort reassembles first, then checks rule
# Fragment 1 (1 byte) + Fragment 2 (1 byte) -> reassembled = 2 bytes -> dsize:>7 = FALSE
# Still evades, but for different reason

# With frag3 DISABLED — Snort sees each fragment as-is
# Fragment (1 byte data) -> dsize:>7 = FALSE -> No alert
```

8. Security Recommendations

#	Recommendation	Priority
1	Keep frag3 preprocessor enabled in snort.conf — never disable it in production. This is the primary defence against IP fragmentation evasion.	**Critical**
2	Remove dsize restriction from ICMP rules. Use: alert icmp any any -> $HOME_NET any (msg:"ICMP Echo"; itype:8; sid:1000001; rev:3;) to catch all ICMP echo requests regardless of payload size.	**High**
3	Add a fragmentation detection rule: alert ip any any -> $HOME_NET any (msg:"Fragmented IP"; fragbits:M; sid:1000002; rev:1;)	**High**
4	Add a tiny payload probe detection rule: alert icmp any any -> $HOME_NET any (msg:"Tiny ICMP Probe"; itype:8; dsize:<8; sid:1000003; rev:1;)	**High**
5	Regularly audit all custom Snort rules for over-restrictive conditions that create exploitable detection gaps.	**Medium**
6	Deploy Snort in inline IPS mode rather than passive IDS mode — block suspicious traffic rather than only alerting on it.	**Medium**

9. Conclusion

Assessment 11C was completed successfully. The exercise demonstrated a complete end-to-end IDS evasion workflow in a fully isolated virtual lab. Starting from a fresh Ubuntu Server installation, Snort 2.9.20 was deployed, configured with a custom ICMP detection rule, and validated against standard ping traffic before the evasion technique was applied.

The evasion was achieved by combining two techniques: a detection rule with an exploitable payload size threshold (dsize:>7) and hping3's IP fragmentation capability (-f -d 1) to send ICMP packets with only 1 byte of payload. Timestamped evidence confirmed zero Snort alerts during the hping3 execution window,

while the same Snort instance was actively detecting normal pings immediately before. This proves the IDS was operational and the evasion was genuine.

Task	Action	Status
Phase 1	Ubuntu 24.04 installed, SSH enabled, connectivity verified	**Completed**
Phase 2	Snort 2.9.20 installed on Ubuntu via apt, ens33 configured	**Completed**
Phase 3	ICMP rule written in local.rules (itype:8, dsize:>7, SID:1000001)	**Completed**
Phase 4	Normal ping from Kali triggered SID:1000001 on every packet	**Completed**
Phase 5	hping3 -1 -f -d 1 sent 5 packets — zero Snort alerts — evasion proven	**Completed**
Report	Full methodology documented with screenshots and analysis	**Completed**

— Sai Aditya | Anonymous India Ethical Hacking Intern | March 2026